

Lab 1:

The screenshot shows a PostgreSQL query editor interface. The query is as follows:

```
--lab 1
SELECT
  relname AS object_name,
  pg_size_pretty(pg_table_size(fed.oid)) AS data_size,
  pg_size_pretty (pg_indexes_size(fed.oid)) AS index_size,
  pg_size_pretty(pg_total_relation_size(fed.oid)) AS total_size
FROM pg_class fed
JOIN pg_namespace n ON n.oid = fed.relnamespace
WHERE fed.relkind = 'r' AND n.nspname = 'public'
ORDER BY pg_total_relation_size(fed.oid) DESC;
```

The results are displayed in a table with the following columns: object_name, data_size, index_size, and total_size. The results are sorted by total_size in descending order.

	object_name	data_size	index_size	total_size
1	rental	1232 kB	1120 kB	2352 kB
2	payment	896 kB	920 kB	1816 kB
3	film	736 kB	200 kB	936 kB
4	film_actor	272 kB	216 kB	488 kB
5	inventory	232 kB	208 kB	440 kB
6	customer	96 kB	112 kB	208 kB
7	address	88 kB	64 kB	152 kB
8	film_category	72 kB	40 kB	112 kB
9	city	64 kB	48 kB	112 kB
10	actor	40 kB	32 kB	72 kB
11	store	8192 bytes	32 kB	40 kB
12	staff	16 kB	16 kB	32 kB
13	rental_log	16 kB	16 kB	32 kB
14	language	8192 bytes	16 kB	24 kB
15	country	8192 bytes	16 kB	24 kB
16	category	8192 bytes	16 kB	24 kB

Total rows: 16 Query complete 00:00:04.190 LF Ln 10, Col 47

The payment and customer tables are the most obvious "Index Heavy" tables in the dvdrental database because the storage dedicated to their indexes exceeds the storage for their actual data.

Lab 2:

```

11
12 --lab 2
13 SELECT pg_size_pretty(pg_indexes_size('payment'));

```

Data Output Messages Notifications



	pg_size_pretty text
1	920 kB

```

15
16 --Step 2: Create a heavy index on payment_date
17 CREATE INDEX idx_payment_date_long ON payment (payment_date, amount, customer_id);

```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 5 secs 510 msec.

```

18
19 --Step 3: Check size again
20 SELECT pg_size_pretty ( pg_indexes_size ( 'payment ' ) ) ;

```

Data Output Messages Notifications



	pg_size_pretty text
1	1488 kB

- The initial index size was **920 kB**.
- After creating the new index, the size became **1488 kB**.
- The new index consumed **568 kB** of disk space (1488 - 920).
- Since each page in PostgreSQL is exactly 8 KB, the new index consumed **71 pages** (568 / 8).

Lab 3:

```

22  --lab 3
23  -- Create Partial Index
24  CREATE INDEX idx_expensive_films
25  ON film (replacement_cost)
26  WHERE replacement_cost > 25.00;

```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 5 secs 120 msec.

```

27
28  -- Find the size of this specific index
29  SELECT relname, pg_size_pretty(pg_relation_size(relid))
30  FROM pg_stat_user_indexes
31  WHERE relname = 'idx_expensive_films';
32

```

Data Output Messages Notifications



relname	pg_size_pretty
name	text

```

32
33  SELECT pg_size_pretty(pg_relation_size('idx_expensive_films'));

```

Data Output Messages Notifications



pg_size_pretty
text
1 16 kB

```

34
35  CREATE INDEX idx_full_replacement_cost ON film (replacement_cost);

```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 9 secs 40 msec.

```
37 SELECT pg_size_pretty(pg_relation_size('idx_full_replacement_cost'));
```

Data Output Messages Notifications

	pg_size_pretty text
1	40 kB

- **Partial Index Size** (idx_expensive_films): **16 kB**.
- **Full Index Size** (idx_full_replacement_cost): **40 kB**.

Conclusion: Using a Partial Index reduced the storage requirement from 40 kB down to 16 kB, saving 60% of the disk space. This practically demonstrates the core benefits of a Partial Index. By using a WHERE clause to filter exactly which rows get indexed, you achieve huge storage savings. Furthermore, querying this index is highly efficient because a smaller tree depth results in faster traversal.